UTEC
Universidad Tecnológica

Algoritmos de ordenamiento

Ordenamiento por selección (Selection Sort)

El ordenamiento por selección no tiene una implementación directa en las colecciones de Java ya que es menos eficiente comparado con los métodos de ordenamiento ya disponibles. Sin embargo, puedes implementar tu propio método de selección usando una lista de `ArrayList`.

```
import java.util.ArrayList;
import java.util.List;

public class SelectionSortExample {
    public static void selectionSort(List<Integer> list) {
        int n = list.size();
        for (int i = 0; i < n - 1; i++) {
            int minIndex = i;
            for (int j = i + 1; j < n; j++) {
                if (list.get(j) < list.get(minIndex)) {
                    minIndex = j;
                }
            }
            int temp = list.get(minIndex);
            list.set(minIndex, list.get(i));
            list.set(i, temp);
        }
    }

    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();
        list.add(64);
        list.add(25);
        list.add(12);
        list.add(22);
        list.add(11);
        selectionSort(list);
        System.out.println(list);
    }
}
```

UTEC
Universidad Tecnológica

Ordenamiento por inserción (Insertion Sort)

Similar al ordenamiento por selección, el ordenamiento por inserción no tiene una implementación directa en las colecciones de Java, pero se puede implementar usando `ArrayList`.

```
import java.util.ArrayList;
import java.util.List;

public class InsertionSortExample {
    public static void insertionSort(List<Integer> list) {
        int n = list.size();
        for (int i = 1; i < n; ++i) {
            int key = list.get(i);
            int j = i - 1;
            while (j >= 0 && list.get(j) > key) {
                list.set(j + 1, list.get(j));
                j = j - 1;
            }
            list.set(j + 1, key);
        }
    }

    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();
        list.add(12);
        list.add(11);
        list.add(13);
        list.add(5);
        list.add(6);
        insertionSort(list);
        System.out.println(list);
    }
}
```

UTEC
Universidad Tecnológica

Heapsort

Para implementar Heapsort, utilizamos la clase `PriorityQueue` de Java que está basada en un heap binario.

```
import java.util.ArrayList;
import java.util.List;
import java.util.PriorityQueue;

public class HeapSortExample {
    public static List<Integer> heapSort(List<Integer> list) {
        PriorityQueue<Integer> heap = new PriorityQueue<>(list);
        List<Integer> sortedList = new ArrayList<>();
        while (!heap.isEmpty()) {
            sortedList.add(heap.poll());
        }
        return sortedList;
    }

    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();
        list.add(4);
        list.add(10);
        list.add(3);
        list.add(5);
        list.add(1);
        List<Integer> sortedList = heapSort(list);
        System.out.println(sortedList);
    }
}
```

Quicksort

El algoritmo Quicksort se puede implementar en una lista de `ArrayList` mediante recursión.

```
import java.util.ArrayList;
import java.util.List;

public class QuickSortExample {
    public static void quickSort(List<Integer> list, int low, int high)
```

UTEC
Universidad Tecnológica

```
{
        if (low < high) {
            int pi = partition(list, low, high);
            quickSort(list, low, pi - 1);
            quickSort(list, pi + 1, high);
        }
    }

    private static int partition(List<Integer> list, int low, int high)
{
        int pivot = list.get(high);
        int i = (low - 1);
        for (int j = low; j < high; j++) {
            if (list.get(j) < pivot) {
                i++;
                int temp = list.get(i);
                list.set(i, list.get(j));
                list.set(j, temp);
            }
        }
        int temp = list.get(i + 1);
        list.set(i + 1, list.get(high));
        list.set(high, temp);
        return i + 1;
    }

    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(7);
        list.add(8);
        list.add(9);
        list.add(1);
        list.add(5);
        quickSort(list, 0, list.size() - 1);
        System.out.println(list);
    }
}
```

UTEC
Universidad Tecnológica

Mergesort

El Mergesort también se puede implementar en una lista de `ArrayList` mediante recursión.

```
import java.util.ArrayList;
import java.util.List;

public class MergeSortExample {
    public static void mergeSort(List<Integer> list, int left, int
right) {
        if (left < right) {
            int mid = (left + right) / 2;
            mergeSort(list, left, mid);
            mergeSort(list, mid + 1, right);
            merge(list, left, mid, right);
        }
    }

    private static void merge(List<Integer> list, int left, int mid, int
right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;

        List<Integer> L = new ArrayList<>();
        List<Integer> R = new ArrayList<>();

        for (int i = 0; i < n1; ++i) {
            L.add(list.get(left + i));
        }
        for (int j = 0; j < n2; ++j) {
            R.add(list.get(mid + 1 + j));
        }

        int i = 0, j = 0;
        int k = left;
        while (i < n1 && j < n2) {
            if (L.get(i) <= R.get(j)) {
                list.set(k, L.get(i));
                i++;
            } else {
                list.set(k, R.get(j));
```

UTEC
Universidad Tecnológica

```
                j++;
            }
            k++;
        }

        while (i < n1) {
            list.set(k, L.get(i));
            i++;
            k++;
        }

        while (j < n2) {
            list.set(k, R.get(j));
            j++;
            k++;
        }
    }

    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();
        list.add(12);
        list.add(11);
        list.add(13);
        list.add(5);
        list.add(6);
        mergeSort(list, 0, list.size() - 1);
        System.out.println(list);
    }
}
```